BANK MANAGEMENT SYSTEM

SQL Database Project Report

Designed & Developed using MySQL Workbench

Submitted By:
NARENDRAVEL H
Academic Year: 2025–2026

1. Introduction

The Bank Management System is a relational database project developed using MySQL. It simulates core banking operations including customer management, account handling, transaction processing, branch management, and loan tracking. The project demonstrates the practical use of SQL concepts such as table creation, data insertion, joins, views, and update operations.

This report provides a comprehensive overview of the database design, schema structure, sample data, SQL operations performed, and the query outputs obtained during execution.

2. Project Objectives

- Design a normalized relational database for a banking system.
- Create and manage five core tables: Customers, Branch, Accounts, Transactions, and Loans.
- Insert realistic sample data for testing and demonstration purposes.
- Perform JOIN operations to retrieve combined information across tables.
- Implement UPDATE statements to simulate real-time balance changes.
- Create SQL Views for simplified querying of customer account details.
- Track loan details associated with each customer.

3. Database Design & Schema

The database comprises five interrelated tables. The entity-relationship model links Customers to both Accounts and Loans, while Accounts are linked to both Branch and Transactions. Foreign key constraints enforce referential integrity throughout.

3.1 Entity-Relationship Overview

- Customers (1) → Accounts (Many): One customer can hold multiple accounts.
- Branch (1) → Accounts (Many): One branch can manage multiple accounts.
- Accounts (1) → Transactions (Many): One account can have multiple transactions.
- Customers (1) → Loans (Many): One customer can have multiple loans.

3.2 Full Schema Reference

Table Name	Column	Data Type	Description
Customers	customer_id	INT (PK)	Unique customer identifier
	name	VARCHAR(50)	Customer full name
	address	VARCHAR(100)	Residential address

	phone	VARCHAR(15)	Contact phone number
	email	VARCHAR(50)	Email address
Branch	branch_id	INT (PK)	Unique branch identifier
	branch_name	VARCHAR(50)	Name of the branch
	location	VARCHAR(50)	City/location of branch
Accounts	account_id	INT (PK)	Unique account identifier
	customer_id	INT (FK)	References Customers table
	branch_id	INT (FK)	References Branch table
	account_type	VARCHAR(20)	Savings or Current
	balance	DECIMAL(10,2)	Current account balance
Transactions	transaction_id	INT (PK)	Unique transaction identifier
	account_id	INT (FK)	References Accounts table
	transaction_type	VARCHAR(20)	Deposit or Withdraw
	amount	DECIMAL(10,2)	Transaction amount
	transaction_date	DATE	Date of transaction
Loans	loan_id	INT (PK)	Unique loan identifier
	customer_id	INT (FK)	References Customers table
	loan_amount	DECIMAL(10,2)	Loan amount sanctioned
	loan_type	VARCHAR(50)	Home / Car / Personal / Education
	loan_status	VARCHAR(20)	Active or Closed

4. Table Creation (DDL)

The following SQL statements were used to create the five tables with appropriate data types, primary keys, and foreign key constraints:

4.1 Customers Table

```
CREATE TABLE Customers (
  customer_id INT PRIMARY KEY,
  name        VARCHAR(50),
  address     VARCHAR(100),
  phone       VARCHAR(15),
  email       VARCHAR(50)
```

```
);
```

4.2 Branch Table

```
CREATE TABLE Branch (
  branch_id   INT PRIMARY KEY,
  branch_name VARCHAR(50),
  location    VARCHAR(50)
);
```

4.3 Accounts Table

```
CREATE TABLE Accounts (
  account_id   INT PRIMARY KEY,
  customer_id  INT,
  branch_id    INT,
  account_type VARCHAR(20),
  balance      DECIMAL(10,2),
  FOREIGN KEY(customer_id) REFERENCES Customers(customer_id),
  FOREIGN KEY(branch_id)   REFERENCES Branch(branch_id)
);
```

4.4 Transactions Table

```
CREATE TABLE Transactions (
  transaction_id   INT PRIMARY KEY,
  account_id       INT,
  transaction_type VARCHAR(20),
  amount           DECIMAL(10,2),
  transaction_date DATE,
  FOREIGN KEY(account_id) REFERENCES Accounts(account_id)
);
```

4.5 Loans Table

```
CREATE TABLE Loans (
  loan_id     INT PRIMARY KEY,
  customer_id INT,
  loan_amount DECIMAL(10,2),
  loan_type   VARCHAR(50),
  loan_status VARCHAR(20),
  FOREIGN KEY(customer_id) REFERENCES Customers(customer_id)
);
```

5. Sample Data Inserted

5.1 Customers

Cust ID	Name	City	Phone	Email
101	Arun	Chennai	9876543210	arun@gmail.com
102	Rahul	Delhi	9876543211	rahul@gmail.com

103	Priya	Mumbai	9876543212	priya@gmail.com

5.2 Accounts (with Balance Updates)

Acc ID	Cust ID	Branch ID	Type	Balance (INR)
2001	101	1	Savings	15,000.00 → 9,000.00 (after updates)
2002	102	2	Current	20,000.00
2003	103	1	Savings	12,000.00 → 15,000.00 (after credit)

5.3 Transactions

Txn ID	Acc ID	Type	Amount (INR)	Date
1	2001	Deposit	5,000.00	2026-01-10
2	2001	Withdraw	2,000.00	2026-01-12
3	2002	Deposit	7,000.00	2026-01-11

5.4 Loans

Name	Loan ID	Amount (INR)	Loan Type	Status
Arun	1	2,00,000.00	Home Loan	Active
Arun	2	50,000.00	Personal Loan	Active
Rahul	3	1,50,000.00	Car Loan	Closed
Rahul	5	75,000.00	Personal Loan	Active
Priya	4	1,00,000.00	Education Loan	Active

6. SQL Operations Performed

6.1 Displaying Customer Account Details (JOIN)

A INNER JOIN between Customers and Accounts retrieves each customer's account type and balance in a single result set:

```
SELECT Customers.name, Accounts.account_type, Accounts.balance
FROM Customers
JOIN Accounts ON Customers.customer_id = Accounts.customer_id;
```

Result: Displays Name, Account Type, and Balance for Arun (Savings – 15,000), Rahul (Current – 20,000), and Priya (Savings – 12,000) as shown in the output screenshots.

6.2 Updating Account Balances (DML — UPDATE)

Two UPDATE statements were executed to simulate a credit transaction and a debit transaction:

```
-- Credit INR 3,000 to account 2003 (Priya)
UPDATE Accounts SET balance = balance + 3000 WHERE account_id = 2003;

-- Debit INR 2,000 from account 2001 (Arun)
UPDATE Accounts SET balance = balance - 2000 WHERE account_id = 2001;
```

After execution, Priya's balance increased to INR 15,000 and Arun's balance decreased to INR 9,000, as confirmed by the subsequent SELECT * FROM Accounts query output.

6.3 Transaction History for a Specific Account

```
SELECT * FROM Transactions WHERE account_id = 2001;
```

This query returned two records for account 2001: a Deposit of INR 5,000 on 2026-01-10 and a Withdrawal of INR 2,000 on 2026-01-12.

6.4 Creating a View for Customer Balances

A SQL VIEW named customer_accounts was created to simplify balance lookups:

```
CREATE VIEW customer_accounts AS
SELECT Customers.name, Accounts.balance
FROM Customers
JOIN Accounts ON Customers.customer_id = Accounts.customer_id;

SELECT * FROM customer_accounts;
```

The view returns a clean two-column result: Name and Balance, making it easy to check balances without writing a full JOIN query each time.

6.5 Loan Tracking Query (JOIN)

```
SELECT Customers.name, Loans.loan_id, Loans.loan_amount,
       Loans.loan_type, Loans.loan_status
FROM Customers
JOIN Loans ON Loans.customer_id = Customers.customer_id;
```

This query joins the Customers and Loans tables to display full loan details against each customer. Arun has two active loans (Home Loan: INR 2,00,000 and Personal Loan: INR

50,000), Rahul has one closed Car Loan and one active Personal Loan, and Priya has one active Education Loan.

7. Query Output Summary

The following table summarises the key query outputs observed during project execution, as visible in the attached screenshots:

Query / Operation	Output / Observation
SELECT * FROM Customers	3 rows: Arun (Chennai), Rahul (Delhi), Priya (Mumbai) with phone and email.
JOIN: Customer + Account details	Arun – Savings – 15,000 | Rahul – Current – 20,000 | Priya – Savings – 12,000
UPDATE balance +3000 (Acc 2003)	Priya's balance updated to INR 15,000.
UPDATE balance -2000 (Acc 2001)	Arun's balance reduced to INR 9,000.
Transaction history (Acc 2001)	2 records: Deposit INR 5,000 on 10-Jan & Withdrawal INR 2,000 on 12-Jan 2026.
SELECT * FROM customer_accounts (VIEW)	Arun – 9,000 | Rahul – 20,000 | Priya – 15,000
Loan tracking JOIN	5 loan records displayed across 3 customers with loan type and status details.

8. Key SQL Concepts Demonstrated

Concept	Usage in This Project
DDL — CREATE TABLE	Defined 5 tables with appropriate column types and constraints.
DML — INSERT	Populated all tables with realistic sample data.
DML — UPDATE	Modified account balances to simulate real banking transactions.
DQL — SELECT	Retrieved data with filters (WHERE) and full table scans.
JOIN (INNER JOIN)	Combined Customers+Accounts and Customers+Loans for rich reports.
CREATE VIEW	Built a reusable view for simplified customer balance queries.
Primary Key (PK)	Enforced uniqueness on all identifier columns.
Foreign Key (FK)	Maintained referential integrity across related tables.
DECIMAL Data Type	Used for precise monetary values in balance and loan amounts.

9. Conclusion

The Bank Management System SQL project successfully demonstrates the design and implementation of a relational database for core banking operations. All five tables were created with proper constraints and populated with sample data. The project covered a wide range of SQL operations including DDL, DML, DQL, JOIN queries, and VIEW creation.

The system is capable of tracking customer accounts, monitoring transaction history, viewing real-time balances through a SQL view, and displaying comprehensive loan information. This project provides a solid foundation that can be extended to include authentication modules, stored procedures, triggers, and a front-end application interface in future iterations.

10. References

- MySQL 8.0 Reference Manual — https://dev.mysql.com/doc/
- W3Schools SQL Tutorial — https://www.w3schools.com/sql/
- MySQL Workbench User Guide — https://dev.mysql.com/doc/workbench/en/
- Course notes and lab materials provided by the instructor.